The Journal of Open Source Software

Mesa 3: Agent-based modeling with Python in 2025

Ewout ter Hoeven [1], **Jan Kwakkel** [1], **Vincent Hess** [2], **Thomas Pike** [3], **Boyu Wang** [4], **rht** [5], **and Jackie Kazil** [3]

1 Delft University of Technology (Faculty of Technology, Policy and Management), the Netherlands **2** Independent Researcher, Germany **3** George Mason University (Department of Computational Social Science), United States **4** University at Buffalo (Department of Geography), United States **5** Independent Researcher

DOI: 10.21105/joss.07668

Software

- Review
- Repository
- Archive

Editor: George K. Thiruvathukal

Reviewers:

- @martibosch
- @jofmi

Submitted: 30 December 2024
Published: 28 March 2025

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Mesa is an open-source Python framework for agent-based modeling (ABM) that enables researchers to create, analyze, and visualize agent-based simulations. Mesa provides a comprehensive set of tools and abstractions for modeling complex systems, with capabilities spanning from basic agent management to sophisticated representation of spaces where agents interact. First released in 2014 and published in Masad et al. (2015) (with updates published in Kazil et al. (2020)), this paper highlights advancements and presents Mesa in its current version (3.1.5) as of 2025.

Statement of need

Agent-based models (ABMs) are composed of autonomous, heterogeneous agents interacting locally with other agents. These interactions give rise to emergent phenomena. The aggregate dynamics of a system under study emerge from these local interactions (Joshua M. Epstein, 1999; Joshua M. Epstein & Axtell, 1996). This type of modeling quickly grew more sophisticated, requiring frameworks to execute them. This led to the establishment of NetLogo in 1999 and MASON in 2003.

NetLogo is the most widely adopted tool and the first to make ABMs accessible, but it only allows for small models. MASON is Java-based, allowing for advancements in scalability and speed above NetLogo, but MASON is difficult for non-programmers. Both of these tools did not serve models over HTTP, which allows for hosting models on the web, nor did they take advantage of the rich scientific Python ecosystem. In response to these needs, Mesa was created with the goal of accessibility – targeting both beginner and advanced programmers. The major release of Mesa 3 provides advanced usability and stabilized functionality. These features include enhanced management of agents, data collection advancements, an improved visualization framework, and making it easier for researchers to create and analyze complex simulations.

Applications

Since its creation in 2014, Mesa has been applied to modeling a wide range of phenomena from economics and sociology to ecology and epidemiology and has been cited in more than 500 papers and 800 authors. Mesa has been applied across diverse domains, including:

- Infrastructure resilience and post-disaster recovery planning (Sun & Zhang, 2020)
- Market modeling, including renewable energy auctions and consumer behavior (Anatolitis & Welisch, 2017)

The Journal of Open Source Software

主题三：2025 年基于 Python 的代理建模

埃沃特·特·霍芬[1]，**Jan Kwakkel**[1]，**文森特·赫斯**[2]，**托马斯·派克**[3]，**王博宇**[4]，**rht**[5]，以及 **Jackie Kazil**[3]

1 荷兰代尔夫特理工大学（技术、政策与管理学院）**2**德国独立研究员**3**美国乔治梅森大学（计算社会科学系）**4**美国布法罗大学（地理学系）**5**独立研究员

DOI：10.21105/joss.07668**软件**

- 评审
- 代码仓库
- 档案

编辑：乔治·K·蒂鲁瓦图卡尔

审稿人：

- @martibosch
- @jofmi

提交日期：2024年12月30日
发布日期：2025年3月28日

许可证
论文作者保留版权，并依据《知识共享 署名 4.0 国际许可协议》（CC BY 4.0）发布本作品。

摘要

Mesa 是一个基于代理的建模（ABM）开源 Python 框架，助力研究人员创建、分析并可视化基于代理的模拟。Mesa 提供了一套全面的工具与抽象机制，用于建模复杂系统，其功能涵盖从基础的代理管理到代理交互空间的精细表示。该框架首次发布于 2014 年，并在 Masad 等人（2015）的论文中发布（后续更新见 Kazil 等人（2020）的论文），本文重点介绍了该框架的最新进展，并展示了截至 2025 年的当前版本（3.1.5）。

需求说明

基于智能体的模型（ABMs）由自主、异质的智能体构成，这些智能体在局部范围内与其他智能体进行交互。此类交互会引发涌现现象。所研究系统的整体动力学特性正是源于这些局部交互（Joshua M. Epstein，1999；Joshua M. Epstein & Axtell，1996）。这类建模方法迅速日趋复杂，亟需相应的框架来支撑其运行。这推动了NetLogo于1999年以及mason于2003年的问世。

NetLogo 是最广泛采用的工具，也是首个使 Agent-Based Modeling (ABM) 变得易于使用的工具，但它仅支持小型模型。mason 基于 Java 构建，在可扩展性和运行速度方面较 NetLogo 有所提升，但对于非编程人员而言操作难度较高。这两种工具均不支持通过HTTP协议托管模型（该协议可实现模型在 Web 上的部署），也未充分利用 Python 丰富的科学计算生态系统。为满足这些需求，Mesa 应运而生，其核心目标在于提升易用性——同时面向初级与高级编程人员。Mesa 3 的主要版本提供了更先进的易用性和稳定的功能，包括增强的智能体管理功能、数据收集能力的提升、优化的可视化框架，以及助力研究人员更便捷地创建与分析复杂仿真模型。

应用场景

自 2014 年问世以来，Mesa 已被广泛应用于从经济学、社会学到生态学及流行病学等诸多领域的现象建模，并被超过 500 篇论文及 800 位作者引用。Mesa 已在以下多个不同领域中得到应用：

- 基础设施韧性及灾后恢复规划（Sun & Zhang，2020）
- 市场建模，包括可再生能源拍卖与消费者行为（Anatolitis& Welisch，2017）

- Transportation optimization, such as combined truck-drone delivery routing (Leon-Blanco et al., 2022)
- Recommender systems analysis examining consumer-business value tradeoffs over time (Ghanem et al., 2022)
- Climate adaptation modeling examining household-level behavioral responses to environmental shocks (Taberna et al., 2023)
- SEIR modeling of Sars-CoV-2 (Covid-19) (Pham et al., 2021)
- Management of edge computing resources (Souza et al., 2023)

These applications showcase Mesa's versatility in modeling complex systems with autonomous interacting agents, whether representing individual consumers, infrastructure components, buildings, or vehicles.

The framework is particularly suited for:

- Models with heterogeneous agent populations
- Systems requiring sophisticated spatial interactions
- Interactive exploration of parameter spaces
- Teaching and learning agent-based modeling

Core capabilities

Mesa is a Python-based framework for ABM that provides a comprehensive set of tools for creating, running, and analyzing ABMs. Mesa integrates with the wider scientific Python ecosystem with libraries such as NumPy, pandas, Matplotlib, NetworkX, and more. The backend of the framework is written in Python, while the front-end end uses a Python implementation of React. The modular architecture is comprised of three main components:

1. Core ABM components (*i.e.,* agents, spaces, agent activation, control over random numbers) to build models
2. Data collection and support for model experimentation
3. Visualization systems

This decoupled design allows selective use of components while enabling extension and customization.

Mesa follows a two-track development model where new features are first released as experimental before being stabilized. Experimental features are clearly marked as such and may have their APIs change between releases. They are graduated to stable status once their APIs and implementations are proven through community testing and feedback. Stable features follow semantic versioning.

Core ABM components

Model

The central class in Mesa is the Model. To build a model, the user instantiates a model object, creates a space within it, and populates the space with agent instances. Since ABMs are typically stochastic simulations, Mesa includes a random number generator and, for reproducibility purposes, allows the user to pass a seed.

```
class SimpleModel(mesa.Model):
    def __init__(self, n_agents=10, seed=42):
        super().__init__(seed=seed)  # Initialize Mesa model with random seed

        SimpleAgent.create_agents(self, n_agents, energy=100)
```

- 运输优化，例如卡车与无人机联合配送路径规划（Leon-Blanco et al.，2022）
- 推荐系统分析：考察消费者与企业价值权衡随时间变化的情况（Ghanem 等，2022）
- 针对家庭层面对环境冲击的行为反应进行的气候适应建模研究（Taberna 等，2023）
- SARS-CoV-2（COVID-19）的 SEIR 模型（Pham 等，2021）
- 边缘计算资源管理（Souza 等，2023）

这些应用充分展现了 Mesa 在利用自主交互智能体建模复杂系统方面的多功能性——无论这些智能体代表的是单个消费者、基础设施组件、建筑物还是车辆。

本框架尤其适用于以下场景：

- 具有异质性智能体群体的模型
- 需要复杂空间交互作用的系统
- 参数空间的交互式探索
- 基于代理的建模教学与学习

核心能力

Mesa 是一个基于 Python 的 ABM 框架，为创建、运行及分析 ABM 提供了一套完整的工具集。Mesa 可与更广泛的科学 Python 生态系统集成，支持使用诸如NumPy、pandas、Matplotlib、NetworkX等库。该框架的后端采用 Python 编写，前端则采用基于 Python 实现的 React。其模块化架构包含以下三个核心组件：

1. 构建模型所需的核心 ABM 组件（*即*智能体、空间、智能体激活机制以及对随机数的控制）
2. 数据收集及模型实验支持
3. 可视化系统

这种解耦式设计支持对组件进行选择性使用，同时便于扩展与定制。

Mesa 采用双轨开发模式：新功能首先以实验性功能形式发布，随后才进入稳定版本；实验性功能均明确标注，且其 API 可能在不同版本间发生变化；一旦其 API 与实现已通过社区测试及反馈验证，这些功能便会晋升为稳定版本；稳定功能则遵循语义化版本控制规范。

核心 ABM 组件

模型

Mesa 的核心类是 Model。要构建一个模型，用户需要实例化一个模型对象，在其内部创建一个空间，并将该空间填充以智能体实例。由于行为建模（ABM）通常是基于随机性的模拟，Mesa 内置了一个随机数生成器，并且出于可复现性的考虑，允许用户传入一个种子值。

```
class SimpleModel(mesa.Model):
                                    def init ( self，n_agents=10，seed=42):
        super ( ). init (seed= seed)# 使用随机种子初始化 Mesa 模型

        SimpleAgent.create_agents( self，n_agents，energy=100)
```

```
    def step(self):
        self.agents.shuffle_do("step")  # Activate all agents in random order
```

Agents

Central to ABMs are the autonomous heterogeneous agents. Mesa provides a variety of base agent classes which the user can subclass. In its most basic implementation, an agent subclass specifies the `__init__` and `step` method. Any subclass of the basic mesa agent subclass registers itself with the specified model instance, and via `agent.remove` it will remove itself from the model. It is strongly encouraged to rely on `remove`, and even extend it if needed to ensure agents are fully removed from the simulation. Sometimes an agent subclass is referred to as a "type" of agent.

```
class SimpleAgent(mesa.Agent):
    def __init__(self, model, energy):
        super().__init__(model)  # Initialize Mesa agent
        self.energy = energy

    def step(self):
        self.energy -= 1
        if self.energy <= 0:
            self.remove()
```

Agent management

One significant advancement of Mesa 3 is expanded functionality around agent management. The new `AgentSet` class provides methods that allow users to filter, group, and analyze agents, making it easier to express complex model logic.

When agents are created, they automatically register with the model via `model.register_agent(self)`. This registration automatically adds the agent to an `AgentSet` that's accessible through the model's `agents` property. Additional `AgentSet` instances for each agent type are maintained and available through `model.agents_by_type`. These collections are automatically updated when agents are added or removed from the model.

```
# Select wealthy agents and calculate average wealth
wealthy = model.agents.select(lambda a: a.wealth > 1000)
avg_wealth = wealthy.agg("wealth", func=np.mean)

# Group agents by type and apply behaviors
grouped = model.agents.groupby("species")
for species, agents in grouped:
    agents.shuffle_do("reproduce")
```

Spaces

Mesa 3 provides both discrete (cell-based) and continuous space implementations. In discrete spaces, an agent occupies a cell. Mesa implements discrete spaces using a doubly-linked structure where each cell maintains connections to its neighbors. The framework includes several discrete space variants with a consistent API:

- Grid-based: `OrthogonalMooreGrid`, `OrthogonalVonNeumanGrid`, and `HexGrid`
- Network-based: `Network` for graph-based topologies
- Voronoi-based: `VoronoiMesh` for irregular tessellations (where space is divided into cells based on proximity to seed points)

Example grid creation:

```
grid = OrthogonalVonNeumannGrid((width, height), torus=False, random=model.random)
```

```
    def step(self):
        self.agents.shuffle_do ("step") # 以随机顺序激活所有代理
```

代理

ABM 的核心是自主异质智能体。Mesa 提供了多种基础智能体类，用户可基于这些类进行子类化。在最基础的实现中，智能体子类需定义 `init` 与 `step`_方法。 任何基于基础 Mesa 智能体类创建的子类，均须向指定的模型实例注册自身；而通过 `agent.remove()` 方法，则可将该子类从模型中移除。强烈建议采用 `remove()` 方法，若需确保智能体被完全从仿真环境中移除，甚至可扩展该方法。有时，智能体子类亦被称为该智能体的"类型"。

```
class SimpleAgent(mesa.Agent):
    def init ( self， model， energy):
        super ( ) . init (model)# 初始化 Mesa 智能体
        自身.能量=能量

    def step(self):
        自身.能量=-1
        if self.energy <= 0:
            self.remove( )
```

代理管理

Mesa 3 的一项重要改进是扩展了代理管理相关功能。全新的`AgentSet`类提供了多种方法，使用户能够对代理进行筛选、分组和分析，从而更便捷地表达复杂的模型逻辑。

当创建智能体时，它们将通过 `model.register_agent(self)` 方法自动向模型注册。此注册操作会自动将该智能体添加至一个可通过模型 `agents` 属性访问的 `AgentSet` 集合。针对每种智能体类型，系统还会维护额外的 `AgentSet` 实例，这些实例可通过 `model.agents_by_type` 获取。当智能体在模型中被添加或移除时，这些集合亦会自动更新。

```
                                    # 筛选高净值代理人并计算其平均财富值
富有= model.agents.select ( lambda a: a.wealth >1000)
avg_wealth = wealthy.agg("wealth"， func= np.mean)


                                    # 按类型对代理进行分组并应用相关行为
grouped = model.agents.groupby( "species")
对于各物种，相关代理按以下方式分组:
    agents.shuffle_do("复制")
```

空格

Mesa 3 同时提供离散空间（基于单元格）与连续空间的实现方案。在离散空间中，智能体占据一个单元格。Mesa 采用双链表结构实现离散空间，其中每个单元格均与其相邻单元格建立连接。该框架包含多种具有统一 API 的离散空间变体：

- 基于网格的：OrthogonalMooreGrid、OrthogonalVonNeumanGrid以及HexGrid
- 基于网络的：用于图拓扑结构的网络
- 基于 Voronoi 的方法：VoronoiMesh，用于不规则镶嵌（其中空间根据与种子点的邻近程度被划分为若干单元格）

网格创建示例：

```
grid= OrthogonalVonNeumannGrid((width，height)，torus= False，random= model . random)
```

在 Mesa 3 中，新增了用于离散空间内

In Mesa 3, specialized agent classes for spatial interactions in discrete spaces were added:

- `FixedAgent`: Is assigned to a cell, can access this cell, but cannot move to another cell.
- `CellAgent`: Can move between cells
- `Grid2DMovingAgent`: Extends `CellAgent` with directional movement methods

All discrete spaces support PropertyLayers - efficient numpy-based arrays for storing cell-level properties. This newly added feature allows for agents to interact with spatial properties of the cell more easily:

```
grid.create_property_layer("elevation", default_value=10)
high_ground = grid.elevation.select_cells(lambda x: x > 50)
```

For models where agents need to move continuously through space rather than between discrete locations, `ContinuousSpace` allows agents to occupy any coordinate within defined boundaries:

```
space = ContinuousSpace(x_max, y_max, torus=True)
space.move_agent(agent, (new_x, new_y))
```

Time advancement

Mesa supports two primary approaches to advancing time in simulations: incremental-time progression (tick-based) and next-event time progression.

Typically, ABMs represent time in discrete steps (often called "ticks"). For each tick, the model's step method is called, and agents are activated to take their designated actions. The most frequently implemented approach is shown below, which runs a model for 100 ticks:

```
model = Model(seed=42)

for _ in range(100):
    model.step()
```

Before Mesa 3, all agents were activated within the step method of the model using predefined schedulers. However, the newly added `AgentSet` class provides a more flexible way to activate agents. These changes include the removal of the Scheduler API and its previously available fixed patterns.

```
model.agents.do("step")  # Sequential activation

model.agents.shuffle_do("step")  # Random activation

# Multi-stage activation:
for stage in ["move", "eat", "reproduce"]:
    model.agents.do(stage)

# Activation by agent subclass:
for klass in model.agent_types:
    model.agents_by_type[klass].do("step")
```

Mesa also includes experimental support for next-event time progression through the `DiscreteEventSimulator`. This experimental feature allows scheduling events at arbitrary timestamps rather than fixed ticks, enabling both pure discrete event-based models and hybrid approaches. The latter hybrid approach combines traditional ABM time steps with the flexibility and potential performance benefits of event scheduling. While currently experimental, this capability is being actively developed and tested:

```
# Pure event-based scheduling (experimental)
simulator = DiscreteEventSimulator()
model = Model(seed=42, simulator=simulator)
simulator.schedule_event_relative(some_function, 3.1415)
```

空间交互的专用智能体类：

- FixedAgent：被分配至某个单元格，可访问该单元格，但无法移动至其他单元格。
- CellAgent：可在细胞间移动
- Grid2DMovingAgent：为CellAgent扩展出定向移动方法

所有离散空间均支持 PropertyLayers —— 这是一种基于 NumPy 的高效数组结构，用于存储细胞级属性。这一新添加的功能使智能体能够更便捷地与细胞的空间属性进行交互：

```
grid.create_property_layer( “elevation”, default_value=10)
high_ground = grid.elevation.select_cells(lambda x: x > 50)
```

对于智能体需要在连续空间中而非离散位置之间移动的模型，ContinuousSpace允许智能体占据定义边界内的任意坐标：

```
space = ContinuousSpace (x_max, y_max, torus= True)
space.move_agent (agent, (new_x, new_y))
```

时间推进

Mesa 支持两种推进模拟时间推进的主要方法：增量时间推进（基于时间步长）以及下一事件时间推进。

通常，ABM 以离散时间步长（常称为“tick”）表示时间。在每个 tick 时段内，模型会调用其步进算法，并激活智能体以执行其指定动作。以下所示为最常用的实现方式，该方式运行模型 100 个 tick：

```
模型 = 模型 (种子值=42)

对于 _in范围(100):
    模型。step()
```

在 Mesa 3 之前，所有代理均通过模型中的步骤方法并使用预定义的调度器来激活。然而，新添加的AgentSet类提供了一种更灵活的代理激活方式。这些变更包括移除 Scheduler API 及其此前可用的固定模式。

```
model.agents.do (“step”) # 顺序激活

model.agents.shuffle_do( “step”)# 随机激活

# 多阶段激活：
针对阶段在[“移动”，“进食”，“繁殖”]中：
    model.agents.do（阶段）

# 由药物亚类激活：
对于模型中的类klass，其类型为 model(agent_types)：
    model.agents_by_type[klass] .do(“step”)
```

Mesa 还通过DiscreteEventSimulator提供了针对下一事件时间推进的实验性支持。这一实验性功能允许在任意时间戳而非固定时间步长上调度事件，从而支持纯基于离散事件的模型以及混合式模型。后者将传统 ABM 的时间步长与事件调度所提供的灵活性及潜在性能优势相结合。尽管目前仍处于实验阶段，但该功能正在积极开发与测试中：

```
# 纯基于事件的调度（实验性）
模拟器 = DiscreteEventSimulator()
model = 模型 (seed=42, simulator= simulator)
simulator.schedule_event_relative (some_function, 3.1415)
```

```
# Hybrid incremental time and next-event time progression (experimental)
model = Model(seed=42, simulator=ABMSimulator())
model.simulator.schedule_event_next_tick(some_function)
```

Visualization

Mesa's visualization module, SolaraViz, allows for interactive browser-based model exploration. Advancements with Mesa 3 update the visualization from harder-to-maintain custom code to Solara, a standardized library. Usage of the visualization module can be seen below:

```
visualization = SolaraViz(
    model=model,
    components=[
        make_space_component(wolf_sheep_portrayal),          # Grid visualization
        make_plot_component(["Wolves", "Sheep", "Grass"]),   # Population plot
        lambda m: f"Step {m.steps}: {len(m.agents)} agents"  # Text display
    ],
    model_params=model_params
)
```

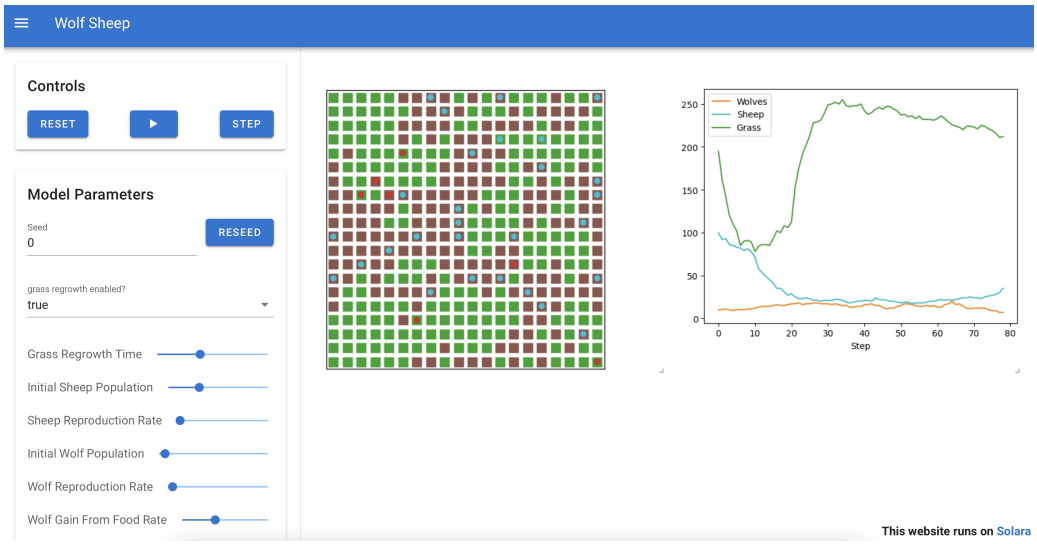


Figure 1: A screenshot of the WolfSheep Model in Mesa

Key features include:

- Interactive model controls
- Real-time data visualization
- Customizable agent and space portrayal
- Support for multiple visualization types, including grids, networks, and charts

Experimentation and analysis

Data collection

Mesa's DataCollector enables systematic data gathering during simulations:

```
collector = DataCollector(
    model_reporters={"population": lambda m: len(m.agents)},
    agent_reporters={"wealth": "wealth"},
```

```
# 混合增量时间与下一事件时间推进法（实验性）
model = Model (seed=42，simulator=ABMSimulator())
model.simulator.schedule_event_next_tick(some_function)
```

可视化

Mesa 的可视化模块SolaraViz支持基于浏览器的交互式模型探索。Mesa 3 的演进已将可视化功能从难以维护的定制代码升级为Solara——一个标准化库。该可视化模块的用法如下所示：

```
可视化 = SolaraViz(
    模型=模型，
    组件= [
        make_space_component（狼与绵羊肖像）                    # 网格可视化
        make_plot_component([“狼”，“绵羊”，“草地”])，# 种群分布图lambda m: f“阶梯式”{ m . steps}: { len (m.
        agents)}个智能体“# 文本显示
    ]，
    model_params = model_params
```

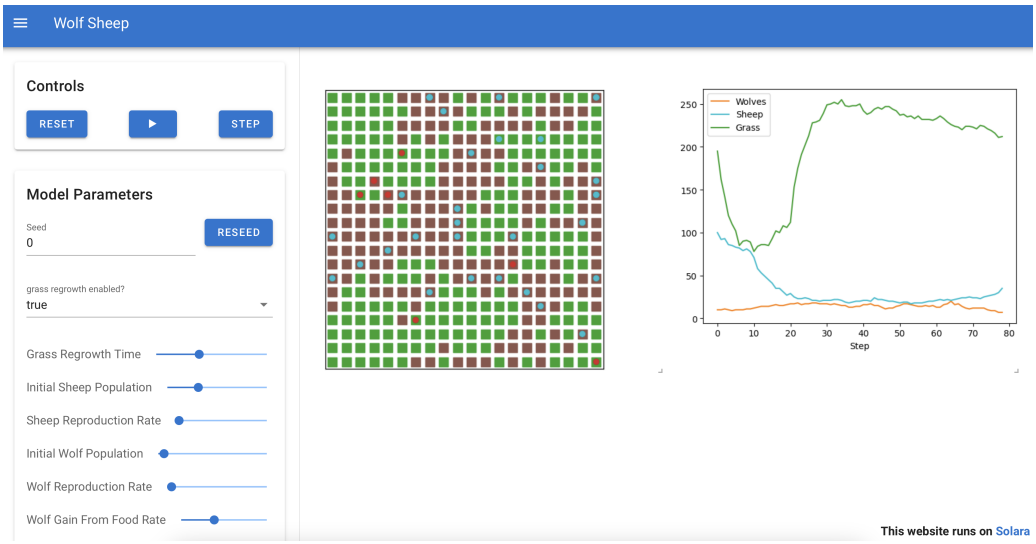


图 1： Mesa 中 WolfSheep 模型的截图

主要特性包括：

- 交互式模型控件
- 实时数据可视化
- 可自定义代理与空间呈现
- 支持多种可视化类型，包括网格图、网络图及图表。

实验与分析

数据收集

Mesa 的 DataCollector 可在仿真过程中实现系统性数据采集：

```
收集器 = DataCollector(
    model_reporters={“人口数”: lambda m: len (m.agents)}，agent_reporters={“财富”:
    “wealth”}，
```

```
        agenttype_reporters={
            Predator: {"kills": "kills_count"},
            Prey: {"distance_fled": "flight_distance"}
        }
    )
```

The collected data integrates seamlessly with pandas for analysis:

```
model_data = collector.get_model_vars_dataframe()
agent_data = collector.get_agent_vars_dataframe()
```

Parameter sweeps

Mesa supports systematic parameter exploration:

```
parameters = {
    "num_agents": range(10, 100, 10),
    "growth_rate": [0.1, 0.2, 0.3]
}
results = mesa.batch_run(MyModel, parameters, iterations=5, max_steps=10)
```

Community and ecosystem

Mesa has grown into a complete ecosystem with extensions including:

- Mesa-Geo for geospatial modeling (Wang et al., 2022)
- Mesa-Frames for high-performance simulations
- A rich collection of community-contributed extensions, example models, and tutorials

Conclusions

Mesa 3 introduces significant advancements to the Python ABM framework, enhancing the core toolkit with greater control, interactivity, and speed for researchers. These notable improvements, paired with its foundational integration with the scientific Python ecosystem, modular architecture, and active community, make it an indispensable tool for researchers across disciplines working in Python who want to create and analyze agent-based models.

Acknowledgements

The advancements leading to Mesa 3 were developed by seven maintainers (the authors) and an active community with over 140 contributors. We would especially like to thank David Masad for his foundational work on Mesa.

References

Anatolitis, V., & Welisch, M. (2017). Putting renewable energy auctions into action–an agent-based model of onshore wind power auctions in Germany. *Energy Policy*, *110*, 394–402. https://doi.org/10.1016/j.enpol.2017.08.024

Epstein, Joshua M. (1999). Agent-based computational models and generative social science. *Complexity*, *4*(5), 41–60.

Epstein, Joshua M., & Axtell, R. (1996). *Growing artificial societies: Social science from the bottom up*. Brookings Institution Press; MIT Press. ISBN: 9780262550253

```
        agenttype_reporters={
                                捕食者：{“kills”：“kills_count”}，
            猎物：{“flight_distance”：“飞行距离”}}
    )
```

所收集的数据可与 pandas 无缝集成，以便进行分析：

```
model_data = collector.get_model_vars_dataframe()
agent_data = collector.get_agent_vars_dataframe()
```

参数扫描

Mesa 支持系统性参数探索：

```
参数 = {
    “num_agents”:范围(10，100，10)
    “growth_rate”:[0.1，0.2，0.3]}
results = mesa.batch_run(MyModel，参数，iterations=5，max_steps=10)
```

社区与生态系统

Mesa 已发展成为一个完整的生态系统，其扩展功能包括：

- Mesa-Geo用于地理空间建模（Wang 等，2022）
- Mesa-Frames：用于高性能模拟
- 丰富的社区贡献扩展程序、示例模型及教程资源

结论

Mesa 3 为 Python 基于智能体的建模（ABM）框架带来了重大革新，通过为研究人员提供更强大的控制能力、更强的交互性及更快的运行速度，显著增强了核心工具集。这些重要改进，加之其与科学 Python 生态系统的深度整合、模块化架构以及活跃的社区支持，使其成为跨学科 Python 研究人员构建与分析基于智能体模型时不可或缺的工具。

致谢

Mesa 3 的演进成果由七位维护者（即本文作者）以及一个拥有超过 140 名贡献者的活跃社区共同开发而成。我们特别感谢David Masad在 Mesa 领域所奠定的基础性工作。

参考文献

Anatolitis, V. , & Welisch, M.(2017).《推动可再生能源拍卖落地——德国陆上风电拍卖的基于智能体的模型》.*《能源政策》*,*110*,394–402. https://doi.org/10.1016/j.enpol.2017.08.024

Epstein，Joshua M. (1999). 基于智能体的计算模型与生成式社会科学。*Complexity*，*4* (5)，41–60.

Epstein，Joshua M. & Axtell，R. (1996)。*《培育人工社会：自下而上的社会科学》*。布鲁金斯学会出版社；麻省理工学院出版社。ISBN：9780262550253

Ghanem, N., Leitner, S., & Jannach, D. (2022). Balancing consumer and business value of recommender systems: A simulation-based analysis. *Electronic Commerce Research and Applications*, *55*, 101195. https://doi.org/10.1016/j.elerap.2022.101195

Kazil, J., Masad, D., & Crooks, A. (2020). Utilizing Python for agent-based modeling: The Mesa framework. *Social, Cultural, and Behavioral Modeling: 13th International Conference, SBP-BRiMS 2020*, 308–317. https://doi.org/10.1007/978-3-030-61255-9_30

Leon-Blanco, J. M., Gonzalez-R, P. L., Andrade-Pineda, J. L., Canca, D., & Calle, M. (2022). A multi-agent approach to the truck multi-drone routing problem. *Expert Systems with Applications*, *195*, 116604. https://doi.org/10.1016/j.eswa.2022.116604

Masad, D., Kazil, J. L., & others. (2015). Mesa: An agent-based modeling framework. *SciPy*, 51–58. https://doi.org/10.25080/Majora-7b98e3ed-009

Pham, T. M., Tahir, H., Wijgert, J. H. H. M. van de, Van der Roest, B. R., Ellerbroek, P., Bonten, M. J. M., Bootsma, M. C. J., & Kretzschmar, M. E. (2021). Interventions to control nosocomial transmission of SARS-CoV-2: A modelling study. *BMC Medicine*, *19*, 211. https://doi.org/10.1186/s12916-021-02060-y

Souza, P. S., Ferreto, T., & Calheiros, R. N. (2023). EdgeSimPy: Python-based modeling and simulation of edge computing resource management policies. *Future Generation Computer Systems*, *148*, 446–459. https://doi.org/10.1016/j.future.2023.06.013

Sun, J., & Zhang, Z. (2020). A post-disaster resource allocation framework for improving resilience of interdependent infrastructure networks. *Transportation Research Part D: Transport and Environment*, *85*, 102455. https://doi.org/10.1016/j.trd.2020.102455

Taberna, A., Filatova, T., Hadjimichael, A., & Noll, B. (2023). Uncertainty in boundedly rational household adaptation to environmental shocks. *Proceedings of the National Academy of Sciences*, *120*(44), e2215675120. https://doi.org/10.1073/pnas.2215675120

Wang, B., Hess, V., & Crooks, A. (2022). Mesa-geo: A GIS extension for the Mesa agent-based modeling framework in Python. *Proceedings of the 5th ACM SIGSPATIAL International Workshop on GeoSpatial Simulation*, 1–10. https://doi.org/10.1145/3557989.3566157

Ghanem，N.、Leitner，S. 与 Jannach，D.（2022）。推荐系统的消费者价值与商业价值平衡：基于仿真的分析。*《电子商务研究与应用》*，*第55卷*，第101195页。https://doi.org/10.1016/j.elerap.2022.101195

Kazil，J.、Masad，D. 与 Crooks，A.（2020）。利用 Python 进行基于智能体的建模：Mesa 框架。*《社会、文化与行为建模：第 13 届国际会议，SBP -BRiMS 2020》*，第 308–317 页。https://doi.org/10.1007/978-3-030-61255-9_30

Leon-Blanco，J. M.、Gonzalez-R，P. L.、Andrade-Pineda，J. L.、Canca，D. 及 Calle，M.（2022）。卡车多无人机路径规划问题的多智能体解决方案。*《专家系统及其应用》*，*第195卷*，第116604页。https://doi.org/10.1016/j.eswa.2022.116604

Masad，D.，Kazil，J. L.，及其他作者。(2015). Mesa：一种基于智能体的建模框架。*SciPy*，51–58. https://doi.org/10.25080/Majora-7b98e3ed-009

Pham，T. M.、Tahir，H.、Wijgert，J. H. H. M. van de、Van der Roest，B. R.、Ellerbroek，P.、Bonten，M. J. M.、Bootsma，M. C. J. 及 Kretzschmar，M. E.（2021）。控制 SARS-CoV-2 院内传播的干预措施：一项模型研究。*BMC Medicine*，*19*，211。https://doi.org/10.1186/s12916-021-02060-y

Souza，P. S.、Ferreto，T. 与 Calheiros，R. N.（2023）。EdgeSimPy：基于 Python 的边缘计算资源管理策略建模与仿真。*《未来世代计算机系统》*，*第 148 卷*，第 446–459 页。https://doi.org/10.1016/j.future.2023.06.013

Sun，J. 与 Zhang，Z.（2020）。一种灾后资源分配框架，旨在提升相互依赖基础设施网络的韧性。*《交通研究D辑：交通与环境》*，*第85卷*，第102455页。https://doi.org/10.1016/j.trd.2020.102455

Taberna，A.、Filatova，T.、Hadjimichael，A. 与 Noll，B.（2023）。有限理性家庭对环境冲击的适应中的不确定性。*《美国国家科学院院刊》*，*120*（44），e2215675120。https://doi.org/10.1073/pnas.2215675120

Wang，B.、Hess，V. 与 Crooks，A.（2022）。Mesa-geo：用于 Python 中 Mesa 基于代理建模框架的 GIS 扩展。*《第5届 ACM SIGSPATIAL 国际地理空间模拟研讨会论文集》*，第 1–10 页。https://doi.org/10.1145/3557989.3566157